AI COMPONENT · CODE & CONCEPTS (BEGINNER LEVEL)

At-Risk Student Detection (ML)

How the platform flags struggling students early — explained from your code

Intelligent Adaptive Learning Platform · Yassine Ben Nessib · EspritTech — explained from the real source code

> **In one sentence —** We compute **8 numbers** describing a student's activity, feed them to a trained classifier, and get a **risk probability** plus the **top factors** behind it. Code: `risk_service.py` (use) + `train_risk_model.py` (training).

1 The idea in one minute

Many students struggle silently. If the platform can **flag** them early, a teacher can help before they drop out. This is an **early-warning** system based on **supervised machine learning**: describe each student with numbers (features), train a model on past examples, predict who is at risk.

2 Where it lives in the code

`app/services/risk_service.py`
Builds the 8 features and runs the prediction.

`scripts/train_risk_model.py`
Trains and evaluates the model.

`app/data/risk_model.joblib`
The saved trained model + scaler.

`scripts/generate_risk_data.py`
Makes the (simulated) training data.

3 Step 1 — building the 8 features from the database

The model needs numbers. `extract_features()` reads the student's real quiz history from the database and computes 8 signals.

RISK_SERVICE.PY — EXTRACT_FEATURES() (EXCERPT)

```
results = db.query(QuizResult).filter(...).all()
scores = [float(r.score) for r in results]
avg_score = sum(scores) / len(scores)            # 1. average score
quizzes_count = len(results)                     # 2. how many quizzes
avg_time_min = sum(r.temps_reponse for r in results)/len(results)/60  # 3. time
days_since_last = (now - last).total_seconds()/86400  # 4. inactivity
score_trend = np.polyfit(xs, scores, 1)[0]       # 5. going up or down?
avg_mastery = sum(mvals)/len(mvals)              # 6. BKT mastery (Brick 1)
low_mastery_ratio = (# concepts below 40) / total# 7. weak concepts
error_rate = wrong / total                       # 8. error rate
```

What this does, step by step:

- It pulls the student's `QuizResult` rows from the database (their quiz history).
- It then computes 8 numbers: average score, number of quizzes, average time, days since last activity, the **trend** of scores (`np.polyfit` fits a line — a negative slope means scores are dropping), average BKT mastery, the share of weak concepts, and the error rate.
- These 8 features are **exactly** the ones the training data uses, so a model trained on simulated data plugs straight into real data.

4 Step 2 — predicting the risk

RISK_SERVICE.PY — PREDICT_FROM_FEATURES()

```
x = np.array([[float(features[f]) for f in order]])  # put the 8 numbers in order
xs = bundle["scaler"].transform(x)                   # standardise them
proba = float(bundle["clf"].predict_proba(xs)[0][1]) # P(at risk)
at_risk = proba >= bundle["threshold"]               # flag if above 0.5
# explain WHY: coefficient x standardised value
contrib = bundle["clf"].coef_[0] * xs[0]
top_factors = [name for name, c in sorted(zip(order, contrib),
               key=lambda kv: -kv[1]) if c > 0][:3]
return {"at_risk": at_risk, "probability": proba, "top_factors": top_factors}
```

What this does, step by step:

- The 8 features are put in the right **order** and standardised by the saved `scaler` (so "80%" and "12 minutes" become comparable).
- `predict_proba(...)[0][1]` gives the probability of the "at-risk" class (column 1).
- If that probability is above the threshold (0.5), the student is flagged `at_risk = True` .
- **Explanation:** `coef_ × value` tells how much each feature pushed this student toward risk; we surface the **top 3 positive factors** — so the teacher sees *why*.

5 Step 3 — how the model was trained

TRAIN_RISK_MODEL.PY — THE TRAINING PIPELINE

```
scaler = StandardScaler().fit(X)            # learn the scale of each feature
Xs = scaler.transform(X)
X_tr, X_te, y_tr, y_te = train_test_split(Xs, y, test_size=0.2, stratify=y)
clf = LogisticRegression(max_iter=2000, class_weight="balanced")
clf.fit(X_tr, y_tr)                          # learn
roc = roc_auc_score(y_te, clf.predict_proba(X_te)[:,1])   # evaluate
bundle = {"scaler": scaler, "clf": clf, "features": features, "threshold": 0.50}
joblib.dump(bundle, OUT)
```

What this does, step by step:

- `StandardScaler` puts every feature on the same scale (mean 0, spread 1) so no feature dominates just because its numbers are bigger.
- `class_weight="balanced"` is important: usually few students are at-risk, so we tell the model to pay more attention to that rare class.
- We evaluate with **ROC-AUC** and a report; for an early-warning tool, **recall** matters most (better a false alarm than a missed student).
- The scaler + classifier + feature order are saved together in the `.joblib` bundle, so the service reproduces the exact same processing.

```
WHY STANDARDISE (COMPARE DIFFERENT UNITS FAIRLY)
standardised value = ( value − mean ) / standard_deviation
```

6 Worked example

A flagged student

Low average score, few attempts, several failed quizzes, dropping trend → the model outputs `probability = 0.82`. Above 0.5 → `at_risk = True`, with `top_factors = ["low_mastery_ratio", "error_rate", "days_since_last"]`. The teacher gets a clear, explained signal.

7 Strengths & limits

Strengths

- Early warning before failure
- Uses data already collected
- Explains the top factors
- Probability lets you tune caution

Limits

- Needs representative data
- Biased data → biased model
- Correlation, not causation
- Threshold is a precision/recall trade-off

Honest note — The model was trained on **synthetic** students (`generate_risk_data.py`), so it demonstrates the method. The honest next steps: train on **real** cohorts, validate, and check **fairness**. Use the flag only as a support signal, never to punish.

8 Q&A

DEFENSE / INTERVIEW QUESTIONS (INCLUDING “WHAT DOES THIS BLOCK DO?”)

Q What are the 8 features and where do they come from?

A Average score, number of quizzes, average time, days since last activity, score trend, average BKT mastery, ratio of weak concepts, error rate — all computed from the student’s quiz history in the database by extract_features().

Q What does predict_from_features() do?

A It orders and standardises the features, gets the at-risk probability, flags the student above the threshold, and computes the top factors that explain the decision.

Q How do you explain why a student is flagged?

A contrib = coefficient × standardised value for each feature; the largest positive contributions are surfaced as the top factors.

Q Why StandardScaler?

A To compare features with different units (percent, minutes, counts) fairly, so the model is not biased by scale.

Q Why class_weight=balanced?

A Because at-risk students are rare (class imbalance); balancing makes the model focus on catching them.

Q How is it evaluated and why recall?

A With accuracy, precision, recall, F1 and ROC-AUC; recall matters most because missing a struggling student is the costly error.

Q Any ethical concern?

A Yes — avoid bias, protect data, and use the flag only to offer help, not to penalise.

9 Key terms

Feature — One number describing the student.

Label — The known outcome (at-risk: 1 / 0).

StandardScaler — Puts features on the same scale.

Logistic Regression — A simple, explainable classifier giving probabilities.

class_weight balanced — Focus more on the rare class.

predict_proba — Probability of each class.

ROC-AUC — How well the model separates the two classes.

Recall — Share of truly at-risk students we catch.